

EE 475 Introduction to Image Processing

Homework 04

Boran Aybak Kilic
Student ID: 2022401207

1. Theory of Histogram Transformation

The process of histogram transformation involves two distinct operations: **Histogram Equalization** and **Histogram Specification**.

Step 1: Histogram Equalization (Mapping to a Uniform PDF)

The first step is to transform the input image's pixel intensities, r , which have a probability density function (PDF) $p_r(r)$, into new intensities, s , that follow a uniform distribution. This process is known as histogram equalization.

The transformation used is the Cumulative Distribution Function (CDF) of the input, scaled by $(L - 1)$:

$$s = T(r) = (L - 1) \int_0^r p_r(w)dw \quad (1)$$

Derivation

We can prove that the resulting PDF, $p_s(s)$, is uniform. For a transformation of random variables $s = T(r)$, the resulting PDF $p_s(s)$ is given by:

$$p_s(s) = p_r(r) \left| \frac{dr}{ds} \right|_{r=T^{-1}(s)} \quad (2)$$

To find $\frac{dr}{ds}$, we first find its inverse, $\frac{ds}{dr}$, by differentiating Equation 1 with respect to r . Using the Fundamental Theorem of Calculus:

$$\frac{ds}{dr} = \frac{d}{dr} \left[(L - 1) \int_0^r p_r(w)dw \right] = (L - 1)p_r(r) \quad (3)$$

Therefore, $\frac{dr}{ds} = \frac{1}{(L-1)p_r(r)}$. Substituting this back into the PDF transformation formula:

$$p_s(s) = p_r(r) \left| \frac{1}{(L - 1)p_r(r)} \right| = \frac{1}{L - 1} \quad (4)$$

This result, $p_s(s) = \frac{1}{L-1}$, is the PDF of a uniform distribution over the range $[0, L - 1]$. This proves that the transformation $T(r)$ "flattens" the original histogram into a uniform one.

The Purpose of the (L-1) Factor

The factor $(L - 1)$ is a crucial **scaling constant**.

- The CDF, $\int_0^r p_r(w)dw$, is a probability. Its output is, by definition, in the normalized range $[0, 1]$.

- Digital image intensities are not stored in the range $[0, 1]$. They are discrete values in the range $[0, L - 1]$ (e.g., $[0, 255]$ for an 8-bit image where $L = 256$).
- Multiplying the CDF by $(L - 1)$ stretches the output range from $[0, 1]$ to $[0, L - 1]$, mapping the probabilities to the full dynamic range of the output image.

Step 2: Histogram Specification (Mapping to a Desired PDF)

The second step, $z = G^{-1}(s)$, transforms the uniform distribution s into a new, *desired* distribution z , which has a target PDF of $p_z(z)$.

This process uses the *inverse* of the target distribution's scaled CDF. Let's define the CDF of the target distribution $p_z(z)$:

$$s' = G(z) = (L - 1) \int_0^z p_z(w) dw \quad (5)$$

Based on the proof from Step 1, we know this transformation $G(z)$ would also create a uniform distribution, s' .

Since both $T(r)$ from Step 1 and $G(z)$ from Step 2 produce the same uniform distribution, we can set them equal:

$$s = s' \implies T(r) = G(z) \quad (6)$$

To find the final pixel value z that maps from the original pixel r , we solve for z :

$$z = G^{-1}(s) = G^{-1}(T(r)) \quad (7)$$

This completes the process: $T(r)$ equalizes the input image to a uniform distribution, and $G^{-1}(s)$ re-maps those uniform values to produce an image with the desired histogram $p_z(z)$.

Discrete Formulation

In practice, images are discrete. The PDF $p_r(r)$ is replaced by a probability mass function (PMF), $p_r(r_k) = \frac{n_k}{MN}$, where n_k is the number of pixels with intensity r_k , and MN is the total number of pixels. The integral becomes a summation:

$$s_k = T(r_k) = (L - 1) \sum_{j=0}^k p_r(r_j) = \frac{(L - 1)}{MN} \sum_{j=0}^k n_j \quad (8)$$

This discrete CDF is precisely what is calculated in the MATLAB code to build the transformation map.

2. MATLAB Code for Histogram Equalization

The following MATLAB code implements the histogram equalization process as a self-contained function, 'perform_equalization'. The logic follows the discrete formula derived in the previous section.

The function's logic is broken down as follows:

- **Initialization:** First, the image dimensions (rows, columns) are extracted using the `size` function, and the total number of pixels is calculated. The number of intensity levels, L , is set to 256 for an 8-bit image.
- **Histogram (PDF):** The `imhist` function is used to compute the histogram of the input image. This gives a 256-element vector containing the raw counts of each intensity level.

- **PMF:** The histogram counts are normalized by dividing by the `total_pixels` to create the Probability Mass Function (PMF).
- **CDF:** The `cumsum` (Cumulative Sum) function is applied to the PMF. This is the discrete equivalent of integration and produces the Cumulative Distribution Function (CDF).
- **Transformation Map:** The core transformation $T(r) = (L - 1) \times \text{CDF}(r)$ is applied. The result is rounded to the nearest integer using `round` to ensure valid, discrete pixel values for the output. This rounding operation is the reason why pdf of equalized images are not exactly uniform.
- **Pixel Mapping:** The function initializes an empty output image using `zeros`. It then iterates over every pixel (i, j) in the input image. For each pixel, it gets the `old_value` (from 0-255) and uses this to find the `new_value` from the `transform_map`. A crucial step is adding 1 to the `old_value` to create the index, as MATLAB arrays are 1-indexed (position 1 corresponds to intensity 0).

```

1 function [output_image, transform_map] = perform_equalization(input_image)
2     % This function performs histogram equalization on a grayscale input image.
3     % It calculates the image's CDF and uses it as a transformation function
4     % to map input intensities to new output intensities, improving contrast.
5     %
6     % ARGS:
7     %     input_image: A 2D matrix (grayscale image) of type uint8.
8     %
9     % RETURNS:
10    %     output_image: The histogram-equalized image (uint8).
11    %     transform_map: The mapping T(r) from old to new pixel values.
12
13    % Ensure the input is grayscale (2D)
14    if ndims(input_image) > 2
15        error('Input image must be grayscale.');
```

```

46     output_image = zeros(rows, cols, 'uint8');
47
48     % Map each pixel from the old intensity to the new one.
49     for i = 1:rows
50         for j = 1:cols
51             % Get the original intensity value (0-255)
52             old_value = input_image(i, j);
53
54             % Find the new value from the map.
55             % We use (old_value + 1) because MATLAB arrays are 1-indexed,
56             % so intensity 0 maps to index 1, and intensity 255 maps to index
256.
57             output_image(i, j) = transform_map(old_value + 1);
58         }
59     end
60
61     % Convert map to uint8 for consistency
62     transform_map = uint8(transform_map);
63
64 end

```

Listing 1: MATLAB function to perform histogram equalization.

3. Applying the Code to the Given Image

The 'perform_equalization' function was applied to the provided image. The original image, shown in Figure 1, exhibits **low contrast**, which is quantitatively confirmed by its histogram. The histogram's mass is concentrated in a narrow band of mid-to-high intensities, indicating that the full dynamic range of the image is not being used. This compression of intensity values is why fine details in the terrain and structures are difficult to distinguish.

After applying histogram equalization, the resulting image in Figure 2 shows a **dramatic improvement in contrast**. The new histogram demonstrates that the pixel intensities are now **spread across the entire dynamic range**, from 0 to 255. This "stretching" of the histogram is the direct result of applying the intensity transformation function $T(r)$ (the scaled CDF), which is plotted in Figure 3. The slope of this transformation curve dictates how the contrast is expanded. Visually, this enhancement makes qualitative features like buildings, airport runways, and the distinction between land and water much clearer and more defined.

It is important to note that the resulting histogram in Figure 2 is not perfectly uniform, as the continuous theory might suggest. This is a direct consequence of working with digital images, which have a **discrete number of intensity levels**. The transformation $T(r)$ maps input intensities to output values that are often fractional. These values must be rounded to the nearest integer (a process called **quantization**) to be stored as a valid pixel. This rounding action causes multiple input levels to map to a single output level, and some output levels to be skipped entirely, creating the characteristic "spikes" and "gaps" seen in the equalized histogram instead of a perfectly flat distribution.

In conclusion, histogram equalization effectively enhances the visual quality and detail discernibility of the image by spreading its intensity values across the full dynamic range. Although

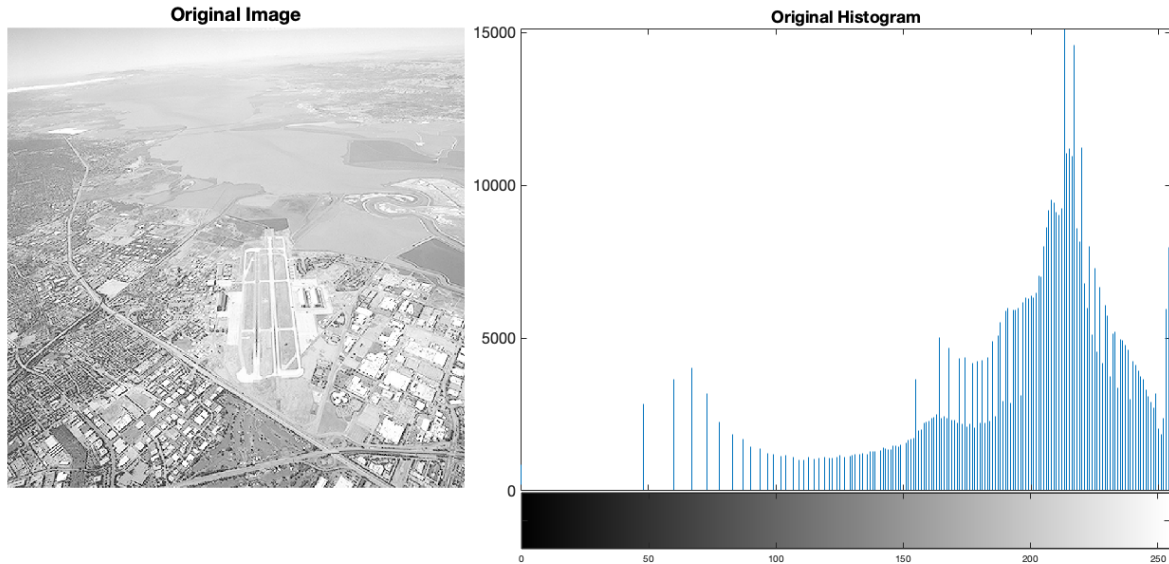


Figure 1: Original Image and its histogram.

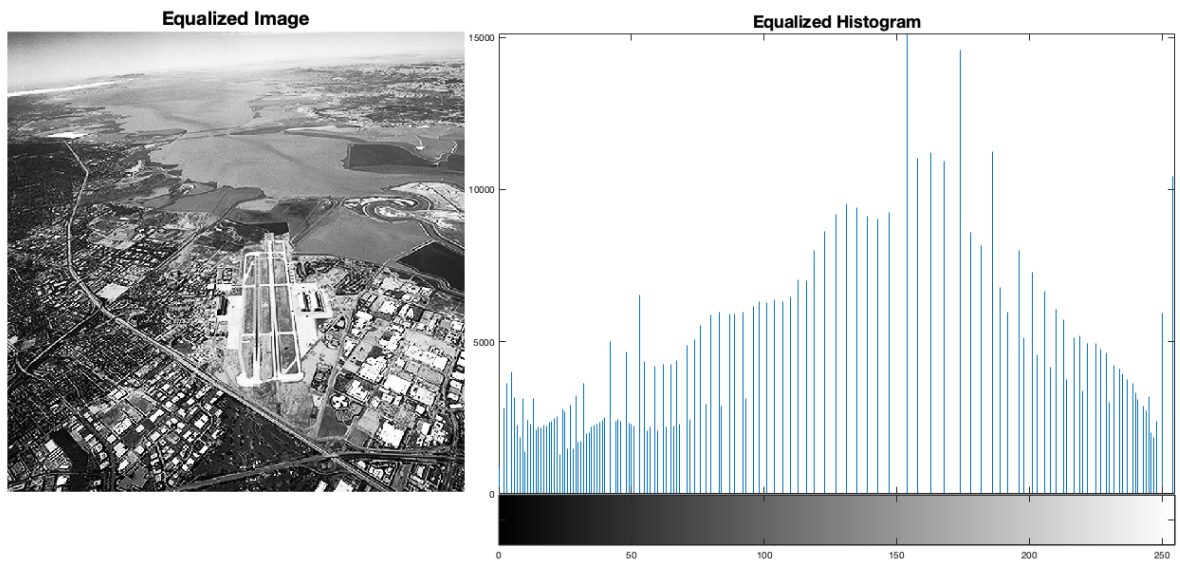
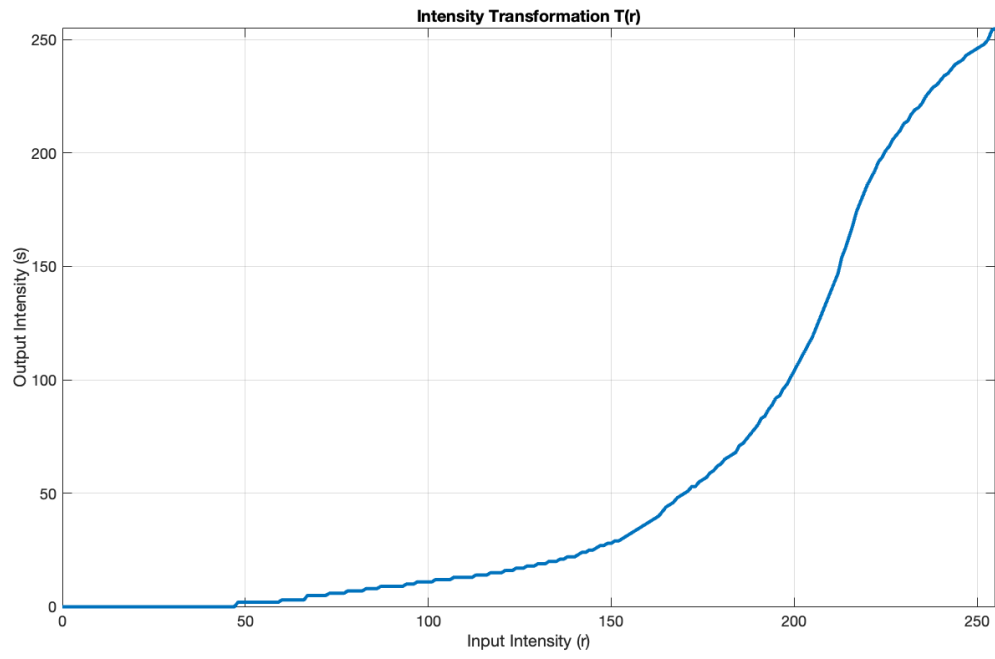


Figure 2: Equalized Image and its histogram.